

LAPORAN PRATIKUM

STRUKTUR DATA

“Pratikum Pekan 8”

Disusun Oleh:

Rafikhul Ramadhan

2511533012

Dosen Pengampu : Dr. Wahyudi, S.T, M.T.

Asisten Praktikum : Muhammad Galis Avero



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Segala puji dan syukur saya panjatkan ke hadirat Allah SWT atas limpahan rahmat, taufik, dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum mata kuliah Struktur Data ini dengan baik dan tepat waktu.

Laporan praktikum ini disusun sebagai salah satu bentuk pemenuhan tugas akademik, sekaligus sebagai sarana untuk memperdalam pemahaman penulis terhadap konsep-konsep dasar dalam struktur data. Melalui praktikum ini, saya mempelajari berbagai materi penting seperti penggunaan array, stack, queue, linked list, serta bagaimana mengimplementasikannya dalam bahasa pemrograman. Selain itu, kegiatan praktikum juga melatih kemampuan berpikir logis, sistematis, dan terstruktur dalam menyelesaikan permasalahan.

Saya menyadari bahwa dalam penyusunan laporan ini masih terdapat berbagai kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, saya sangat mengharapkan kritik dan saran yang membangun guna perbaikan di masa yang akan datang.

Pada kesempatan ini, saya juga ingin menyampaikan rasa terima kasih kepada dosen pengampu, asisten praktikum, serta teman-teman yang telah memberikan bimbingan, dukungan, dan bantuan selama proses praktikum hingga penyusunan laporan ini.

Akhir kata, semoga laporan praktikum ini dapat memberikan manfaat bagi penulis khususnya, serta bagi para pembaca pada umumnya. Penulis berharap laporan ini juga dapat digunakan sebagai referensi dalam memahami materi praktikum, baik pada mata kuliah Struktur Data maupun praktikum lainnya.

Padang, 21 Mei 2026

Rafikhul Ramadhan

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan	1
1.3 Manfaat	1
BAB II PEMBAHASAN.....	2
2.1 Langkah Kerja.....	2
2.2 Shellsort_2511533012.....	2
2.3 Quicksort_2511533012	4
2.4 BubblesortGUI_2511533012	6
2.5 MergesortGUI_2511533012	11
BAB III KESIMPULAN	18
3.1 Ringkasan.....	18
3.2 Kesimpulan.....	18
DAFTAR PUSTAKA	19

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pada praktikum pekan ke-8, mahasiswa mempelajari algoritma sorting lanjutan dalam bahasa pemrograman Java. Berbeda dengan metode sorting dasar yang telah dipelajari sebelumnya, praktikum ini membahas algoritma yang lebih efisien seperti **Quick Sort, Merge Sort, dan Shell Sort**. Selain itu, mahasiswa juga membuat visualisasi proses sorting menggunakan GUI sehingga langkah-langkah pengurutan data dapat diamati secara lebih jelas.

Pemahaman terhadap algoritma sorting lanjutan sangat penting karena sering digunakan dalam pengolahan data berukuran besar yang membutuhkan performa lebih baik dibandingkan metode sorting sederhana.

1.2 Tujuan

Tujuan dari praktikum pekan ke-8 adalah untuk memahami konsep dan cara kerja algoritma Quick Sort, Merge Sort, dan Shell Sort serta mampu mengimplementasikan dan memvisualisasikan proses sorting menggunakan Java.

1.3 Manfaat

Manfaat dari praktikum ini adalah mahasiswa dapat memahami berbagai metode sorting lanjutan, mengetahui kelebihan masing-masing algoritma, serta mampu memilih algoritma yang sesuai untuk kebutuhan pengolahan data tertentu.

BAB II

PEMBAHASAN

2.1 Langkah Kerja

Tahapan yang dilakukan dalam pratikum ini adalah sebagai berikut:

1. Menjalankan aplikasi IDE (seperti Eclipse atau IntelliJ IDEA) atau menggunakan text editor yang mendukung bahasa Java.
2. Membuat sebuah project baru dengan nama **pekan8_2511533012**.
3. Menambahkan tiga buah file Java, yaitu shellsort_2511533012.java, Quicksort_2511533012.java, BubleSortGUI_2511533012.java, MergeSortGUI_2511533012.java.
4. Menuliskan kode program sesuai dengan instruksi yang diberikan pada masing-masing file.
5. Melakukan proses kompilasi menggunakan *Java compiler* agar program dapat dijalankan.
6. Menjalankan program untuk mengamati hasil keluaran sesuai dengan logika yang dibuat.
7. Mengevaluasi hasil eksekusi pada *console* serta menarik kesimpulan dari setiap percobaan yang dilakukan.

2.2 Shellsort_2511533012

```
package pekan8_2511533012;

public class ShellSort_2511533012 {

    public static void shellsort_2511533012(int[] A_3102) {
        int n_3102 = A_3102.length;
        int gap_3102 = n_3102 / 2;
        while (gap_3102 > 0) {
            for (int i_3102 = gap_3102; i_3102 < n_3102;
i_3102++) {
                int temp_3102 = A_3102[i_3102];
                int j_3102 = i_3102;
                while (j_3102 >= gap_3102 && A_3102[j_3102 -
gap_3102] > temp_3102) {
```

```

        A_3102[j_3102] = A_3102[j_3102 -
gap_3102];
        j_3102 = j_3102 - gap_3102;
    }
    A_3102[j_3102] = temp_3102;
}
gap_3102 = gap_3102/2;
}

}

public static void main(String[] args) {
    int[] data_3102 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};

    System.out.print("Sebelum: ");
    printArray_2511533012(data_3102);

    shellsort_2511533012(data_3102);

    System.out.println("sesudah (shellsort): ");
    printArray_2511533012(data_3102);
}

public static void printArray_2511533012(int[] arr_3102) {
    for (int i_3102 : arr_3102)
        System.out.print(i_3102 + " ");
    System.out.println();
}
}

```

Analisis:

Program ini mengimplementasikan algoritma **Shell Sort** untuk mengurutkan data array.

Proses sorting dilakukan dengan membagi data berdasarkan jarak tertentu (*gap*), kemudian melakukan pengurutan pada setiap kelompok data. Nilai gap akan terus diperkecil hingga bernilai satu sehingga seluruh data menjadi terurut. Program menampilkan data sebelum dan sesudah proses sorting.

Kesimpulan: Program ini menunjukkan proses pengurutan data menggunakan metode Shell Sort yang lebih efisien dibandingkan insertion sort biasa.

Output:

```
<terminated> ShellSort_2511533012 [Java Ap  
Sebelum: 3 10 4 6 8 9 7 2 1 5  
sesudah (shellsort):  
1 2 3 4 5 6 7 8 9 10
```

2.3 Quicksort_2511533012

```
package pekan8_2511533012;  
  
public class QuickSort_2511533012 {  
    static void swap_2511533012(int[] arr_3012, int i_3012, int  
j_3012) {  
        int temp_3012 = arr_3012[i_3012];  
        arr_3012[i_3012] = arr_3012[j_3012];  
        arr_3012[j_3012] = temp_3012;  
    }  
  
    static void medianOftrhree_2511533012(int[] arr_3012, int  
low_3012, int high_3012) {  
        int mid_3012 = low_3012 + (high_3012 - low_3012) / 2;  
  
        if (arr_3012[low_3012] > arr_3012[mid_3012]) {  
            swap_2511533012(arr_3012, low_3012, mid_3012);  
        }  
        if (arr_3012[low_3012] > arr_3012[high_3012]) {  
            swap_2511533012(arr_3012, low_3012, high_3012);  
        }  
        if (arr_3012[mid_3012] > arr_3012[high_3012]) {  
            swap_2511533012(arr_3012, mid_3012, high_3012);  
        }  
        swap_2511533012(arr_3012, mid_3012, high_3012);  
    }  
  
    static int patition_2511533012(int[] arr_3012, int low_3012, int  
high_3012) {  
        medianOftrhree_2511533012(arr_3012, low_3012, high_3012);
```

```

        int pivot_3012 = arr_3012[high_3012];
        int i_3012 = low_3012 - 1;

        for (int j_3012 = low_3012; j_3012 <= high_3012 - 1;
j_3012++) {
            if (arr_3012[j_3012] < pivot_3012) {
                i_3012++;
                swap_2511533012(arr_3012, i_3012, j_3012);
            }
        }
        swap_2511533012(arr_3012, i_3012 + 1, high_3012);
        return i_3012 + 1;
    }

    static void Quicksort_2511533012(int[] arr_3012, int low_3012,
int high_3012) {
        if (low_3012 < high_3012) {
            int pi_3012 = patition_2511533012(arr_3012, low_3012,
high_3012);
            Quicksort_2511533012(arr_3012, low_3012, pi_3012 - 1);
            Quicksort_2511533012(arr_3012, pi_3012 + 1, high_3012);
        }
    }

    public static void printArr_2511533012(int[] arr_3012) {
        for (int i_3012 = 0; i_3012 < arr_3012.length; i_3012++) {
            System.out.print(arr_3012[i_3012] + " ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        int[] arr_3012 = {10, 7, 8, 9, 1, 5};
        int N_3012 = arr_3012.length;
        System.out.println("Data sebelum di urutkan: ");
        printArr_2511533012(arr_3012);

        Quicksort_2511533012(arr_3012, 0, N_3012 - 1);

        System.out.println("Data terurut quicsort: ");
        printArr_2511533012(arr_3012);
    }
}

```


Output :

```
<terminated> QuickSort_2511533012 [Java Ap
Data sebelum di urutkan:
10 7 8 9 1 5
Data terurut quicsort:
1 5 7 8 9 10
```

Analisis :

Program ini mengimplementasikan algoritma **Quick Sort** untuk mengurutkan data array.

Proses pengurutan dilakukan dengan memilih sebuah nilai sebagai **pivot**, kemudian membagi data menjadi dua bagian, yaitu data yang lebih kecil dari pivot dan data yang lebih besar dari pivot. Program menggunakan metode *median of three* untuk menentukan pivot sehingga proses pengurutan menjadi lebih optimal. Program menampilkan data sebelum dan sesudah proses pengurutan dilakukan.

Kesimpulan: Program ini menunjukkan implementasi Quick Sort yang bekerja dengan teknik pembagian data dan rekursi.

2.4 BubblesortGUI_2511533012

```
package pekan8_2511533012;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
```

```

import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;

public class BubblesortGUI_2511533012 extends JFrame {
    private static final long serialVersionUID = 1L;
    private int[] array;
    private JLabel[] labelArray;
    private JButton stepButton, resetButton, setButton;
    private JTextField inputField;
    private JPanel panelArray;
    private JTextArea stepArea;

    private int i_3012, j_3012;
    private boolean sorting = false;
    private int stepCount = 1;

    public BubblesortGUI_2511533012() {
        setTitle("Bubble Sort Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        JPanel inputPanel = new JPanel(new FlowLayout());
        inputField = new JTextField(30);
        setButton = new JButton("Set Array");
        inputPanel.add(new JLabel("Masukkan angka (pisahkan dengan koma):"));
        inputPanel.add(inputField);
        inputPanel.add(setButton);

        panelArray = new JPanel();
        panelArray.setLayout(new FlowLayout());

        JPanel controlPanel = new JPanel();
        stepButton = new JButton("Langkah Selanjutnya");
        resetButton = new JButton("Reset");
        stepButton.setEnabled(false);
        controlPanel.add(stepButton);
        controlPanel.add(resetButton);

        stepArea = new JTextArea(8, 60);
        stepArea.setEditable(false);
        stepArea.setFont(new Font("Monospaced", Font.PLAIN, 14));
        JScrollPane scrollPane = new JScrollPane(stepArea);

        add(inputPanel, BorderLayout.NORTH);
    }

```

```

        add(panelArray, BorderLayout.CENTER);
        add(controlPanel, BorderLayout.SOUTH);
        add(scrollPane, BorderLayout.EAST);

        setButton.addActionListener(e -> setArrayFromInput());
        stepButton.addActionListener(e -> performStep_2511533012());
        resetButton.addActionListener(e -> reset_2511533012());
    }

    private void setArrayFromInput() {
        String text_3012 = inputField.getText().trim();
        if (text_3012.isEmpty()) return;
        String[] parts_3012 = text_3012.split(",");
        array = new int[parts_3012.length];
        try {
            for (int i_3012 = 0; i_3012 < parts_3012.length;
i_3012++) {
                array[i_3012] =
Integer.parseInt(parts_3012[i_3012].trim());
            }
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Masukkan hanya
angka!",
                "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }
        i_3012 = 0;
        j_3012 = 0;
        stepCount = 1;
        sorting = true;
        stepButton.setEnabled(true);
        stepArea.setText("");
        panelArray.removeAll();
        labelArray = new JLabel[array.length];
        for (int k_3012 = 0; k_3012 < array.length; k_3012++) {
            labelArray[k_3012] = new
JLabel(String.valueOf(array[k_3012]));
            labelArray[k_3012].setFont(new Font("Arial", Font.BOLD,
24));
            labelArray[k_3012].setOpaque(true);
            labelArray[k_3012].setBackground(Color.WHITE);
            labelArray[k_3012].setBorder(BorderFactory.createLineBorder(Color.BL
ACK));
            labelArray[k_3012].setPreferredSize(new Dimension(50,
50));
            labelArray[k_3012].setHorizontalAlignment(SwingConstants.CENTER);
            panelArray.add(labelArray[k_3012]);
        }
        panelArray.revalidate();
        panelArray.repaint();
    }
}

```

```

private void performStep_2511533012() {
    if (!sorting || i_3012 >= array.length - 1) {
        sorting = false;
        stepButton.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting selesai!");
        return;
    }
    resetHighlights_2511533012();
    StringBuilder stepLog = new StringBuilder();
    labelArray[j_3012].setBackground(Color.CYAN);
    labelArray[j_3012 + 1].setBackground(Color.CYAN);
    if (array[j_3012] > array[j_3012 + 1]) {
        // Swap
        int temp_3012 = array[j_3012];
        array[j_3012] = array[j_3012 + 1];
        array[j_3012 + 1] = temp_3012;
        labelArray[j_3012].setBackground(Color.RED);
        labelArray[j_3012 + 1].setBackground(Color.RED);
        stepLog.append("Langkah ").append(stepCount).append(":
Menukar elemen ke-")
            .append(j_3012).append(" ").append(array[j_3012
+ 1]).append(" dengan ke-")
            .append(j_3012 + 1).append("
").append(array[j_3012]).append(")\n");
    } else {
        stepLog.append("Langkah ").append(stepCount).append(":
Tidak ada pertukaran antara ke-")
            .append(j_3012).append(" dan ke-").append(j_3012
+ 1).append(")\n");
    }
    stepLog.append("Hasil:
").append(arrayToString_2511533012(array)).append("\n\n");
    stepArea.append(stepLog.toString());
    updateLabels_2511533012();
    j_3012++;
    if (j_3012 >= array.length - i_3012 - 1) {
        j_3012 = 0;
        i_3012++;
    }
    stepCount++;
    if (i_3012 >= array.length - 1) {
        sorting = false;
        stepButton.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting selesai!");
    }
}

private void resetHighlights_2511533012() {
    if (labelArray == null) return;
    for (JLabel label : labelArray) {
        label.setBackground(Color.WHITE);
    }
}

```

```

    }

    private void updateLabels_2511533012() {
        for (int k_3012 = 0; k_3012 < array.length; k_3012++) {
labelArray[k_3012].setText(String.valueOf(array[k_3012]));
        }
    }

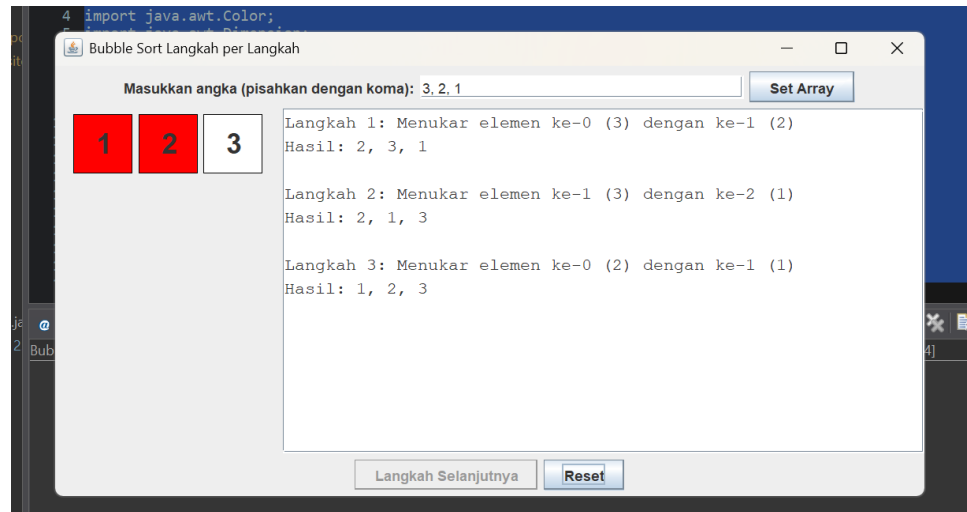
    private void reset_2511533012() {
        inputField.setText("");
        panelArray.removeAll();
        panelArray.revalidate();
        panelArray.repaint();
        stepArea.setText("");
        stepButton.setEnabled(false);
        sorting = false;
        i_3012 = 0;
        j_3012 = 0;
        stepCount = 1;
    }

    private String arrayToString_2511533012(int[] arr_3012) {
        StringBuilder sb = new StringBuilder();
        for (int k_3012 = 0; k_3012 < arr_3012.length; k_3012++) {
            sb.append(arr_3012[k_3012]);
            if (k_3012 < arr_3012.length - 1) sb.append(", ");
        }
        return sb.toString();
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            BubblesortGUI_2511533012 gui = new
BubblesortGUI_2511533012();
            gui.setVisible(true);
        });
    }
}

```

Output :



Analisis :

Program ini merupakan implementasi **Bubble Sort berbasis GUI** menggunakan Java Swing.

Pengguna dapat memasukkan data array melalui form input, kemudian menjalankan proses sorting secara bertahap menggunakan tombol *Langkah Selanjutnya*. Setiap proses perbandingan dan pertukaran elemen ditampilkan secara visual dengan perubahan warna sehingga pengguna dapat memahami proses Bubble Sort dengan lebih mudah. Program juga menyediakan fitur reset untuk mengulang proses sorting dari awal.

Kesimpulan: Program ini membantu memahami cara kerja Bubble Sort melalui visualisasi interaktif.

2.5 MergesortGUI_2511533012

```
package pekan8_2511533012;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
```

```

import java.awt.Font;
import java.util.LinkedList;
import java.util.Queue;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;

public class MergesortGUI_2511533012 extends JFrame {
    private static final long serialVersionUID = 1L;
    private int[] array;
    private JLabel[] labelArray;
    private JButton stepButton, resetButton, setButton;
    private JTextField inputField;
    private JPanel panelArray;
    private JTextArea stepArea;

    private int i_3102, j_3102, k_3102;
    private int left_3102, mid_3102, right_3102;
    private int[] temp_3102;
    private boolean isMerging = false;
    private boolean copying = false;
    private int stepCount = 1;
    private Queue<int[]> mergeQueue = new LinkedList<>();

    public MergesortGUI_2511533012() {
        setTitle("Merge Sort Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        JPanel inputPanel = new JPanel(new FlowLayout());
        inputField = new JTextField(30);
        setButton = new JButton("Set Array");
        inputPanel.add(new JLabel("Masukkan angka (pisahkan dengan koma):"));
        inputPanel.add(inputField);
        inputPanel.add(setButton);

        panelArray = new JPanel();
        panelArray.setLayout(new FlowLayout());

        JPanel controlPanel = new JPanel();
        stepButton = new JButton("Langkah Selanjutnya");

```

```

        resetButton = new JButton("Reset");
        stepButton.setEnabled(false);
        controlPanel.add(stepButton);
        controlPanel.add(resetButton);

        stepArea = new JTextArea(8, 60);
        stepArea.setEditable(false);
        stepArea.setFont(new Font("Monospaced", Font.PLAIN, 14));
        JScrollPane scrollPane = new JScrollPane(stepArea);

        add(inputPanel, BorderLayout.NORTH);
        add(panelArray, BorderLayout.CENTER);
        add(controlPanel, BorderLayout.SOUTH);
        add(scrollPane, BorderLayout.EAST);

        setButton.addActionListener(e ->
setArrayFromInput_251133012());
        stepButton.addActionListener(e -> performStep_2511533012());
        resetButton.addActionListener(e -> reset_2511533012());
    }

    private void setArrayFromInput_251133012() {
        String text_3102 = inputField.getText().trim();
        if (text_3102.isEmpty()) return;
        String[] parts_3102 = text_3102.split(",");
        array = new int[parts_3102.length];
        try {
            for (int i_3102 = 0; i_3102 < parts_3102.length;
i_3102++) {
                array[i_3102] =
Integer.parseInt(parts_3102[i_3102].trim());
            }
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Masukkan hanya
angka!",
                "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }
        labelArray = new JLabel[array.length];
        panelArray.removeAll();
        for (int i_3102 = 0; i_3102 < array.length; i_3102++) {
            labelArray[i_3102] = new
JLabel(String.valueOf(array[i_3102]));
            labelArray[i_3102].setFont(new Font("Arial", Font.BOLD,
24));
            labelArray[i_3102].setOpaque(true);
            labelArray[i_3102].setBackground(Color.WHITE);
            labelArray[i_3102].setBorder(BorderFactory.createLineBorder(Color.BL
ACK));
            labelArray[i_3102].setPreferredSize(new Dimension(50,
50));

```



```

labelArray[i_3102].setHorizontalAlignment(SwingConstants.CENTER);
    panelArray.add(labelArray[i_3102]);
}
mergeQueue.clear();
generateMergeSteps_2511533012(0, array.length - 1);
stepButton.setEnabled(true);
stepArea.setText("");
stepCount = 1;
isMerging = false;
panelArray.revalidate();
panelArray.repaint();
}

private void generateMergeSteps_2511533012(int l_3102, int
r_3102) {
    if (l_3102 >= r_3102) return;
    int m_3102 = (l_3102 + r_3102) / 2;
    generateMergeSteps_2511533012(l_3102, m_3102);
    generateMergeSteps_2511533012(m_3102 + 1, r_3102);
    mergeQueue.add(new int[]{l_3102, m_3102, r_3102});
}

private void performStep_2511533012() {
    resetHighlights_2511533012();

    if (!isMerging && !mergeQueue.isEmpty()) {
        int[] range = mergeQueue.poll();
        left_3102 = range[0];
        mid_3102 = range[1];
        right_3102 = range[2];
        temp_3102 = new int[right_3102 - left_3102 + 1];
        i_3102 = left_3102;
        j_3102 = mid_3102 + 1;
        k_3102 = 0;
        copying = false;
        isMerging = true;
        stepArea.append("Langkah " + stepCount++ +
            ": Mulai merge dari " + left_3102 + " ke " +
right_3102 + "\n");
        return;
    }

    if (isMerging && !copying) {
        if (i_3102 <= mid_3102 && j_3102 <= right_3102) {
            labelArray[i_3102].setBackground(Color.CYAN);
            labelArray[j_3102].setBackground(Color.CYAN);
            if (array[i_3102] <= array[j_3102]) {
                temp_3102[k_3102++] = array[i_3102++];
            } else {
                temp_3102[k_3102++] = array[j_3102++];
            }
        }
    }
}

```

```

        stepArea.append("Langkah " + stepCount++ + ":  

Bandingkan dan salin elemen\n");
        return;
    } else if (i_3102 <= mid_3102) {
        temp_3102[k_3102++] = array[i_3102++];
        stepArea.append("Langkah " + stepCount++ + ": Salin  

sisir kiri\n");
        return;
    } else if (j_3102 <= right_3102) {
        temp_3102[k_3102++] = array[j_3102++];
        stepArea.append("Langkah " + stepCount++ + ": Salin  

sisir kanan\n");
        return;
    } else {
        copying = true;
        k_3102 = 0;
        return;
    }
}

if (copying && k_3102 < temp_3102.length) {
    array[left_3102 + k_3102] = temp_3102[k_3102];
    labelArray[left_3102 +  

k_3102].setText(String.valueOf(temp_3102[k_3102]));
    labelArray[left_3102 +  

k_3102].setBackground(Color.GREEN);
    k_3102++;
    stepArea.append("Langkah " + stepCount++ + ": Tempelkan  

ke array utama\n");
    return;
}

if (copying && k_3102 == temp_3102.length) {
    isMerging = false;
    copying = false;
}

if (mergeQueue.isEmpty() && !isMerging) {
    stepArea.append("Selesai.\n");
    stepButton.setEnabled(false);
    JOptionPane.showMessageDialog(this, "Merge Sort  

selesai!");
}

private void resetHighlights_2511533012() {
    if (labelArray == null) return;
    for (JLabel label : labelArray) {
        label.setBackground(Color.WHITE);
    }
}

private void reset_2511533012() {

```

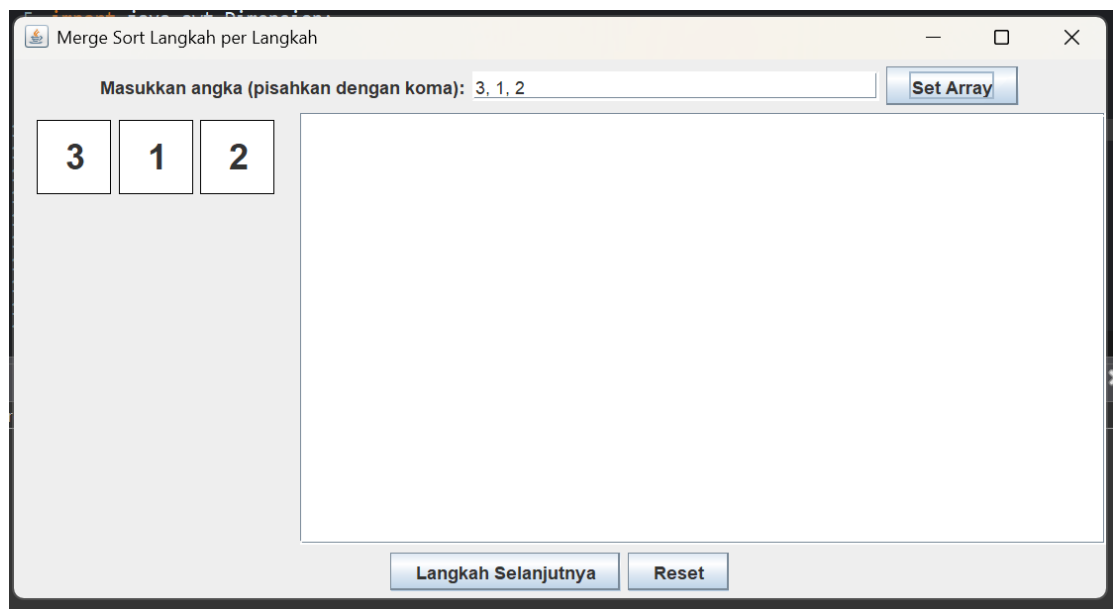
```

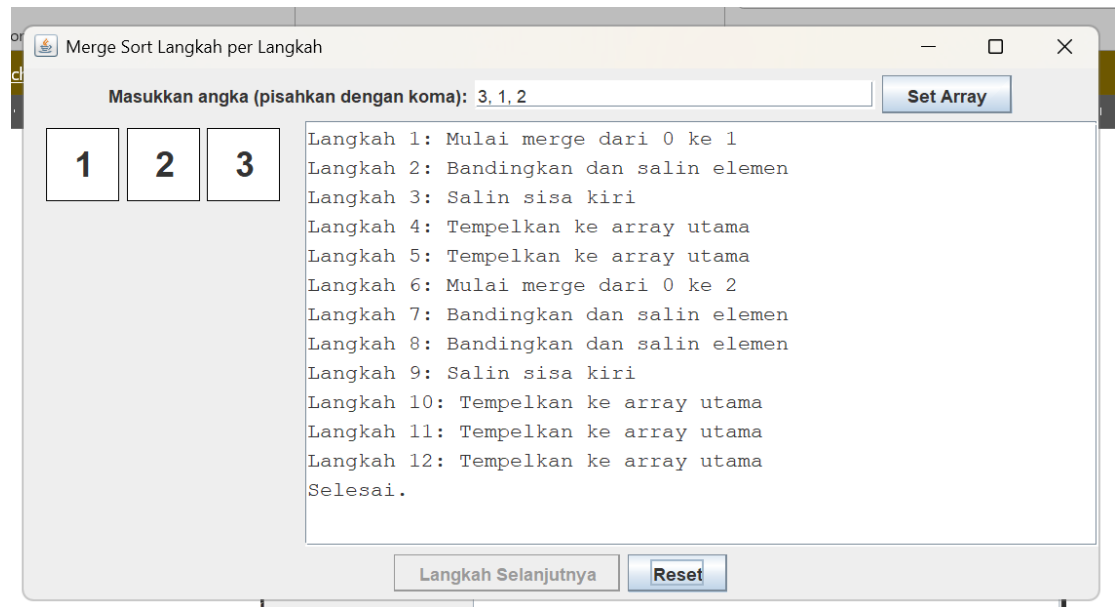
        inputField.setText("");
        panelArray.removeAll();
        panelArray.revalidate();
        panelArray.repaint();
        stepArea.setText("");
        stepButton.setEnabled(false);
        mergeQueue.clear();
        isMerging = false;
        stepCount = 1;
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            MergesortGUI_2511533012 gui = new
MergesortGUI_2511533012();
            gui.setVisible(true);
        });
    }
}

```

Output :





Analisis :

Program ini merupakan implementasi **Merge Sort berbasis GUI** menggunakan Java Swing.

Program membagi array menjadi beberapa bagian kecil, kemudian menggabungkannya kembali dalam keadaan terurut. Setiap langkah proses merge ditampilkan secara bertahap sehingga pengguna dapat melihat bagaimana algoritma Merge Sort bekerja. Selain menampilkan hasil sorting, program juga memberikan informasi setiap langkah proses penggabungan data.

Kesimpulan: Program ini membantu memahami proses pembagian dan penggabungan data pada algoritma Merge Sort secara visual.

BAB III

KESIMPULAN

3.1 Ringkasan

Pada praktikum pekan ke-8, mahasiswa mempelajari implementasi algoritma sorting lanjutan yaitu Quick Sort, Shell Sort, dan Merge Sort. Selain implementasi berbasis console, mahasiswa juga membuat visualisasi sorting menggunakan GUI untuk mempermudah pemahaman terhadap langkah-langkah pengurutan data. Praktikum ini memberikan pemahaman mengenai cara kerja algoritma sorting yang lebih efisien dibandingkan metode sorting dasar.

3.2 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa setiap algoritma sorting memiliki cara kerja dan tingkat efisiensi yang berbeda. Quick Sort menggunakan teknik pembagian data dengan pivot, Merge Sort menggunakan proses divide and conquer, sedangkan Shell Sort menggunakan konsep gap untuk mempercepat proses pengurutan. Dengan adanya visualisasi GUI, mahasiswa dapat lebih memahami proses sorting secara bertahap dan interaktif. Praktikum ini membantu mahasiswa memahami algoritma sorting lanjutan yang banyak digunakan dalam pengolahan data.

DAFTAR PUSTAKA

- [1] Oracle, "The Java Tutorials," 2023. [Online]. Available: <https://docs.oracle.com/javase/tutorial/>.
- [2] H. M. Deitel dan P. J. Deitel, Java: How to Program, 10th ed. Boston: Pearson, 2015

Tugas Pratikum

1. Lagu_2511533012.java

```
package pekan8_2511533012;

public class Lagu_2511533012 {

    String judul_3012;
    String penyanyi_3012;
    int durasi_3012;

    public Lagu_2511533012(String judul_3012, String penyanyi_3012, int
durasi_3012) {
        this.judul_3012 = judul_3012;
        this.penyanyi_3012 = penyanyi_3012;
        this.durasi_3012 = durasi_3012;
    }
}
```

2. Sorting_2511533012

```
package pekan8_2511533012;

public class Sorting_2511533012 {

    Lagu_2511533012[] dataLagu_3012 = new Lagu_2511533012[20];
    int jumlahData_3012 = 0;

    public void inputData_3012() {
        dataLagu_3012[jumlahData_3012++] = new Lagu_2511533012("Hati-
Hati di Jalan", "Tulus", 240);
        dataLagu_3012[jumlahData_3012++] = new
Lagu_2511533012("Monokrom", "Tulus", 220);
        dataLagu_3012[jumlahData_3012++] = new Lagu_2511533012("Sial",
"Mahalini", 210);
        dataLagu_3012[jumlahData_3012++] = new Lagu_2511533012("Tak
Segampang Itu", "Anggi Marito", 250);
        dataLagu_3012[jumlahData_3012++] = new
Lagu_2511533012("Komang", "Raim Laode", 260);
        dataLagu_3012[jumlahData_3012++] = new Lagu_2511533012("Melukis
Senja", "Budi Doremi", 200);
        dataLagu_3012[jumlahData_3012++] = new
Lagu_2511533012("Runtuh", "Feby Putri", 230);
    }

    public void tampilData_3012() {
        for (int i = 0; i < jumlahData_3012; i++) {
            System.out.println(
                (i + 1) + ". " +
                dataLagu_3012[i].judul_3012 + " - " +

```

```

        dataLagu_3012[i].penyanyi_3012 + " - " +
        dataLagu_3012[i].durasi_3012 + " detik"
    );
    }
}

public void quickSort_3012(int low_3012, int high_3012) {
    if (low_3012 < high_3012) {
        int pivotIndex_3012 = partition_3012(low_3012, high_3012);

        quickSort_3012(low_3012, pivotIndex_3012 - 1);
        quickSort_3012(pivotIndex_3012 + 1, high_3012);
    }
}

public int partition_3012(int low_3012, int high_3012) {
    int pivot_3012 = dataLagu_3012[high_3012].durasi_3012;
    int i_3012 = low_3012 - 1;

    for (int j_3012 = low_3012; j_3012 < high_3012; j_3012++) {
        if (dataLagu_3012[j_3012].durasi_3012 < pivot_3012) {
            i_3012++;

            Lagu_2511533012 temp_3012 = dataLagu_3012[i_3012];
            dataLagu_3012[i_3012] = dataLagu_3012[j_3012];
            dataLagu_3012[j_3012] = temp_3012;
        }
    }

    Lagu_2511533012 temp_3012 = dataLagu_3012[i_3012 + 1];
    dataLagu_3012[i_3012 + 1] = dataLagu_3012[high_3012];
    dataLagu_3012[high_3012] = temp_3012;

    return i_3012 + 1;
}

public static void main(String[] args) {

    Sorting_2511533012 playlist_3012 = new Sorting_2511533012();

    playlist_3012.inputData_3012();

    System.out.println("=== Sorting Playlist NIM: 2511533012 ===");

    System.out.println("\nData Sebelum Sorting:");
    playlist_3012.tampilData_3012();

    playlist_3012.quickSort_3012(
        0,
        playlist_3012.jumlahData_3012 - 1
    );
}

```



```

        System.out.println("\nData Setelah Quick Sort (Durasi
Ascending):");
        playlist_3012.tampilData_3012();
    }
}

```

3. Outpun Syntax program

```

<terminated> Sorting_2511533012 [Java Application] C:\Program Files\J
=== Sorting Playlist NIM: 2511533012 ===

Data Sebelum Sorting:
1. Hati-Hati di Jalan - Tulus - 240 detik
2. Monokrom - Tulus - 220 detik
3. Sial - Mahalini - 210 detik
4. Tak Segampang Itu - Anggi Marito - 250 detik
5. Komang - Raim Laode - 260 detik
6. Melukis Senja - Budi Doremi - 200 detik
7. Runtuh - Feby Putri - 230 detik

Data Setelah Quick Sort (Durasi Ascending):
1. Melukis Senja - Budi Doremi - 200 detik
2. Sial - Mahalini - 210 detik
3. Monokrom - Tulus - 220 detik
4. Runtuh - Feby Putri - 230 detik
5. Hati-Hati di Jalan - Tulus - 240 detik
6. Tak Segampang Itu - Anggi Marito - 250 detik
7. Komang - Raim Laode - 260 detik

```

Penjelasan:

Saya memilih algoritma Quick Sort karena memiliki performa yang sangat baik untuk proses pengurutan data dalam jumlah sedang maupun besar. Quick Sort bekerja dengan membagi data berdasarkan sebuah pivot, kemudian mengurutkan bagian kiri dan kanan secara rekursif. Rata-rata kompleksitas waktu Quick Sort adalah $O(n \log n)$, sehingga lebih cepat dibandingkan beberapa algoritma sorting sederhana seperti Bubble Sort atau Insertion Sort. Selain itu, implementasinya cukup efisien karena tidak memerlukan banyak memori tambahan. Oleh karena itu, Quick Sort cocok digunakan untuk mengurutkan data playlist lagu berdasarkan durasi secara ascending.